COMPUTER NETWORKS LAB

MASTER ASSIGNMENT & COMMAND HANDBOOK

Question-wise practical notes, Cisco IOS commands, verification, routing protocols, and socket-programming code

Compiled from the uploaded Week 6, 8, 9, 10, 11, and 12-13 lab assignments

Includes supplementary OSPF and BGP versions of the Week 9 routing topologies

Use this as a revision and command-reference handbook. Source-specific values are kept where the assignments provide them. Values added to fill missing WAN/interface details are clearly labelled as illustrative examples.

Contents

Contents 2

Before You Start: Cisco CLI Quick Reference 5

WEEK 6 - Multilayer Switch, Inter-LAN Routing & Port Security 6

Q1. Configure a 3560 multilayer switch for Inter-LAN routing between VLAN 10 and VLAN 20. 6

Q2. Add PC0-B to PC0 LAN using a separate port and verify communication/MAC learning. 7

Q3. Enable port security on PC0-B port Fa0/3: maximum 1, sticky learning, shutdown violation. 7

Q4. Demonstrate and correct a duplicate IP address. 7

Q5. Compare routed PC0-PC1 traffic with switched PC0-PC0-B traffic. 7

Q6. Configure Cisco 3650 SVIs for PC2 and PC3. 8

Q7. Add PC2-B to VLAN 10 using GigabitEthernet1/0/3. 8

Q8. Enable sticky port security on PC2-B port Gi1/0/3. 8

Q9. Demonstrate duplicate IP on PC2-B and correct it. 9

Q10. Compare PC2-PC3 routed communication with PC2-PC2-B switched communication. 9

Q11. Port security on Switch0 Fa0/1: PC4 authorized, PC6 has same IP but a different MAC. 9

WEEK 6 - Switchport Security, STP, Hub & Collision Domains 10

Q1. Get acquainted with Cisco Packet Tracer. 10

Q2. Configure the three-switch triangle and verify PC0, PC1, PC2 communication. 10

Q3. Enable port security on Switch0 Fa0/3 connected to PC0. 10

Q4. Check Fa0/3 port-security status before and after PC0 sends a packet. 10

Q5. Replace PC0 with a different PC and check port status/violation counter. 11

Q6. Check communication while Fa0/3 is err-disabled. 11

Q7. Recover Fa0/3 and reconnect the authorized PC0. 11

Q8. Check STP status and identify the Root Bridge. 11

Q9. Identify blocking and forwarding ports. 11

Q10. Remove an active triangle link and observe the previously blocked port. 11

Q11. Change Switch1 priority to make it the Root Bridge. 12

Q12. Connect PC0, PC-A, and PC-B to a Hub, then connect the Hub to Switch0 Fa0/3. 12

Q13. Check the switch MAC table for the hub uplink port. 12

Q14. Observe hub flooding in Simulation mode. . . . 12
Q15. Why does a switch learn multiple MAC addresses on one port when a hub is connected? . . . 12
Q16. Compare collision domains: one Hub versus three separate switch ports. . . . 13

WEEK 8 - Static Routing . . . 14

Q1. One router connecting two LANs. . . . 14
Q2. Two routers, one LAN behind each router. . . . 14
Q3. Two routers, three LANs (two LANs on Router0, one LAN on Router1). . . . 15
Q4. Three routers connected in series, four LANs. . . . 15

WEEK 9 - Dynamic Routing: Directly Connected & RIPv2 . . . 17

Q1. One router, two directly connected LANs (no dynamic routing needed). . . . 17
Q2. RIPv2 between two routers and two LANs. . . . 17
Q3. RIPv2 between two routers with three LANs. . . . 18
Q4. RIPv2 between three routers, four LANs and two WAN links. . . . 19

SUPPLEMENT - OSPF Versions of the Week 9 Topologies . . . 21

OSPF for Week 9 Q2 - two routers. . . . 21
OSPF for Week 9 Q3 - two routers, three LANs. . . . 21
OSPF for Week 9 Q4 - three routers, four LANs. . . . 21

SUPPLEMENT - BGP Versions of the Week 9 Topologies . . . 23

BGP for Week 9 Q2 - two routers. . . . 23
BGP for Week 9 Q3 - two routers, three LANs. . . . 23
BGP for Week 9 Q4 - three routers, four LANs. . . . 23

WEEK 10 - VLANs, Trunk vs Access & Router-on-a-Stick . . . 25

Q1. Configure five VLANs on two Cisco 2960 switches and compare trunk mode with access mode. . . . 25
Q2. Router-on-a-Stick using three VLANs and three switches. . . . 26

WEEK 11 - Inter-VLAN Routing with SVIs, ROAS & Trunk Failure . . . 28

Q1. Inter-VLAN routing using SVIs across four access switches and one multilayer switch. . . . 28
Q2. Compare SVI routing on a multilayer switch with Router-on-a-Stick (four VLANs). . . . 30
Q3. VLAN 10 and VLAN 20 across two access switches with a multilayer switch; test trunk failure. . . . 32

WEEK 12 - UDP Socket Programming . . . 35

Q1. UDP Echo Server and Client . . . 35

Q2. UDP Time-of-Day Server and Client . . . 37

WEEK 13 - TCP Socket Programming . . . 40

Q1. TCP String Reversal Server and Client . . . 40

Q2. TCP Arithmetic Calculator Server and Client . . . 42

Final Rapid Revision - Commands to Memorise . . . 48

Before You Start: Cisco CLI Quick Reference

Basic command modes

Prompt / Mode	Command	What it does
Switch> / Router>	enable	Enter privileged EXEC mode.
Switch# / Router#	configure terminal	Enter global configuration mode.
(config)#	interface ...	Enter interface configuration mode.
(config-if)#	exit	Move one level back.
Any config mode	end	Return to privileged EXEC mode.
Switch# / Router#	write memory	Save running configuration to startup configuration.

Useful interface and troubleshooting habits

- Check the exact interface names on the Packet Tracer device before typing commands. Models may show Fa0/1, Gi0/0, Gi1/0/1, or other names.
- Use show ip interface brief to confirm interface names, IP addresses, and up/down state.
- A routed/SVI interface generally needs its VLAN to exist and at least one active Layer 2 port in that VLAN for the SVI line protocol to come up.
- A PC in another IP subnet normally sends traffic to its configured default gateway.
- Use the same subnet mask and gateway values in the PC IP Configuration window as in the addressing table.

Common verification command bank

Command	Use
show vlan brief	VLAN IDs, names, status, and access-port membership.
show ip interface brief	Interface names, IP addresses, and line/protocol status.
show ip route	Complete IPv4 routing table; C=connected, S=static, R=RIP, O=OSPF, B=BGP.
show interfaces trunk	Trunk ports, encapsulation, and VLANs carried.
show mac address-table	MAC addresses learned by a switch and associated ports.
show mac-address-table	Common alternate spelling accepted on some IOS versions.
show port-security interface Fa0/3	Port-security settings, secure MAC count, and violations.
show port-security address	Secure MAC address table.
show interfaces status	Connected, notconnect, and err-disabled port states.
show spanning-tree	Root bridge, root/designated/alternate roles, and port states.
ping IP_ADDRESS	Test IP reachability.
tracert IP_ADDRESS	Trace packet hops from a Packet Tracer PC.

Tip: The first ping may time out while ARP resolves the destination MAC address. Repeat the ping before concluding that the path is broken.

WEEK 6 - Multilayer Switch, Inter-LAN Routing & Port Security

Source assignment: Shreeya Srivastava 20243267 WEEK #6 COMPUTER NETWORKS. This section covers the Cisco 3560/3650 SVI exercises, same-LAN PC additions, port security, duplicate IPs, and routed-versus-switched traffic.

Q1. Configure a 3560 multilayer switch for Inter-LAN routing between VLAN 10 and VLAN 20.

Addressing

Device	IP address	Mask	Default gateway
PC0	192.168.10.10	255.255.255.0	192.168.10.1
PC1	192.168.20.10	255.255.255.0	192.168.20.1
MLS VLAN 10 SVI	192.168.10.1	255.255.255.0	-
MLS VLAN 20 SVI	192.168.20.1	255.255.255.0	-

Multilayer switch commands

```
enable
configure terminal
ip routing

vlan 10
exit
vlan 20
exit

interface vlan 10
 ip address 192.168.10.1 255.255.255.0
 no shutdown
exit

interface vlan 20
 ip address 192.168.20.1 255.255.255.0
 no shutdown
exit

interface gigabitEthernet 1/0/1
 switchport mode access
 switchport access vlan 10
 no shutdown
exit

interface gigabitEthernet 1/0/2
 switchport mode access
 switchport access vlan 20
 no shutdown
exit
end
write memory
```

Interface-name note: The extracted text lists GigabitEthernet1/0/1 and 1/0/2, while one screenshot rendering appears to show FastEthernet0/1 and 0/2. Use the actual ports shown by your Packet Tracer switch.

Verification

```
show vlan brief
show ip interface brief
show ip route
show mac address-table

! From PC0:
ping 192.168.20.10
```

Expected routing table entries: C 192.168.10.0/24 directly connected, Vlan10; C 192.168.20.0/24 directly connected, Vlan20.

Q2. Add PC0-B to PC0 LAN using a separate port and verify communication/MAC learning.

Device	IP	Mask	Gateway
PC0-B	192.168.10.20	255.255.255.0	192.168.10.1

```
enable
configure terminal
interface fastEthernet 0/3
 switchport mode access
 switchport access vlan 10
 no shutdown
exit
end

show vlan brief
show mac address-table

! From PC0:
ping 192.168.10.20
```

Expected VLAN membership: VLAN 10 contains the PC0 port and Fa0/3; VLAN 20 contains the PC1 port. The MAC table learns PC0 and PC0-B on their respective switch ports after they send frames.

Q3. Enable port security on PC0-B port Fa0/3: maximum 1, sticky learning, shutdown violation.

```
enable
configure terminal
interface fastEthernet 0/3
 switchport mode access
 switchport port-security
 switchport port-security maximum 1
 switchport port-security violation shutdown
 switchport port-security mac-address sticky
end

! Verify:
show port-security interface fastEthernet 0/3
show port-security address
```

Connect PC0-B and generate traffic so the switch can learn its sticky secure MAC. Replace it with a different PC and generate traffic. Expected result: Secure-shutdown / err-disabled and violation count increases to 1.

Q4. Demonstrate and correct a duplicate IP address.

Device	Temporary IP	Correct IP
PC0	192.168.10.10	192.168.10.10
PC0-B	192.168.10.10	192.168.10.20

- Temporarily assign 192.168.10.10 to PC0-B, the same address as PC0. Packet Tracer shows a duplicate-IP warning.
- Correct PC0-B to 192.168.10.20, mask 255.255.255.0, gateway 192.168.10.1.
- Verify from PC0 using ping 192.168.10.20.

Q5. Compare routed PC0-PC1 traffic with switched PC0-PC0-B traffic.

Feature	PC0 -> PC1	PC0 -> PC0-B
Network	Different: 192.168.10.0/24 to 192.168.20.0/24	Same: 192.168.10.0/24
OSI layer	Layer 3	Layer 2
Forwarding	SVI/routing table	MAC table
Gateway	Used	Not needed for local forwarding
Useful commands	show ip route; show ip interface brief	show mac address-table

Q6. Configure Cisco 3650 SVIs for PC2 and PC3.

Device	IP	Mask	Gateway
PC2	192.168.10.10	255.255.255.0	192.168.10.1
PC3	192.168.20.10	255.255.255.0	192.168.20.1

```
enable
configure terminal
ip routing
vlan 10
exit
vlan 20
exit

interface vlan 10
 ip address 192.168.10.1 255.255.255.0
 no shutdown
exit
interface vlan 20
 ip address 192.168.20.1 255.255.255.0
 no shutdown
exit

interface gigabitEthernet 1/0/1
 switchport mode access
 switchport access vlan 10
 no shutdown
exit
interface gigabitEthernet 1/0/2
 switchport mode access
 switchport access vlan 20
 no shutdown
exit
end
write memory

show vlan brief
show ip interface brief
show ip route

! From PC2:
ping 192.168.20.10
```

Q7. Add PC2-B to VLAN 10 using GigabitEthernet1/0/3.

PC2-B IP	Mask	Gateway
192.168.10.20	255.255.255.0	192.168.10.1

```
enable
configure terminal
interface gigabitEthernet 1/0/3
 switchport mode access
 switchport access vlan 10
 no shutdown
exit
end

show vlan brief
show mac address-table

! From PC2:
ping 192.168.10.20
```

Q8. Enable sticky port security on PC2-B port Gi1/0/3.

```
enable
configure terminal
interface gigabitEthernet 1/0/3
 switchport mode access
 switchport port-security
 switchport port-security maximum 1
 switchport port-security violation shutdown
 switchport port-security mac-address sticky
end

show port-security interface gigabitEthernet 1/0/3
show port-security address
show interfaces status
```

After PC2-B has generated traffic and its sticky MAC is learned, connect a different PC to Gi1/0/3 and generate traffic. Expected: Secure-shutdown/err-disabled and Security Violation Count = 1.

Q9. Demonstrate duplicate IP on PC2-B and correct it.

- Temporarily set PC2-B to 192.168.10.10, the same IP as PC2.
- Observe the duplicate-IP warning.
- Restore PC2-B to 192.168.10.20/24, gateway 192.168.10.1.
- Verify with ping 192.168.10.20 from PC2.

Q10. Compare PC2-PC3 routed communication with PC2-PC2-B switched communication.

Feature	PC2 -> PC3	PC2 -> PC2-B
Networks	Different (VLAN 10 to 20)	Same (VLAN 10)
Layer	Layer 3	Layer 2
Gateway	Used	Not needed for local forwarding
Table	Routing table	MAC address table
Command	show ip route	show mac address-table

3560 versus 3650: the concepts are the same. The assignment uses different interface naming/examples depending on the model. Always verify actual interface names before entering commands.

Q11. Port security on Switch0 Fa0/1: PC4 authorized, PC6 has same IP but a different MAC.

```
enable
configure terminal
interface fastEthernet 0/1
 switchport mode access
 switchport port-security
 switchport port-security maximum 1
 switchport port-security violation shutdown
 switchport port-security mac-address sticky
end

show port-security interface fastEthernet 0/1
show port-security address
```

- Connect PC4 and generate traffic so its MAC is learned as secure.
- Disconnect PC4 and connect PC6 to the same Fa0/1. PC6 has the same IP address as PC4 but a different MAC address.
- Generate traffic from PC6. Port security checks MAC addresses, so a violation occurs.
- Verify Secure-shutdown/err-disabled and violation count = 1.
- Disconnect PC6, reconnect PC4, and recover the port.

```
enable
configure terminal
interface fastEthernet 0/1
 shutdown
 no shutdown
exit
end

show port-security interface fastEthernet 0/1
show interfaces status
```

Port-security decision is based on MAC addresses, not IP addresses. A different MAC can trigger a violation even if it uses the same IP as the authorized PC.

WEEK 6 - Switchport Security, STP, Hub & Collision Domains

Source assignment: WEEK #6 COMPUTER NETWORK LAB. It contains Questions 1-16 on Packet Tracer, port security, STP, hubs, MAC learning and collision domains.

Q1. Get acquainted with Cisco Packet Tracer.

Cisco Packet Tracer is a network simulation tool used to design, configure, and test networks using simulated routers, switches, PCs, hubs, and Cisco IOS commands. Uses include configuring switches/PCs, testing ping, port security, MAC learning, STP, hubs and collision domains.

Q2. Configure the three-switch triangle and verify PC0, PC1, PC2 communication.

PC	IP address	Mask	Gateway
PC0	192.168.1.10	255.255.255.0	Not required
PC1	192.168.1.20	255.255.255.0	Not required
PC2	192.168.1.30	255.255.255.0	Not required

```
! From PC0:
ping 192.168.1.20
ping 192.168.1.30

! From PC1:
ping 192.168.1.10
ping 192.168.1.30

! From PC2:
ping 192.168.1.10
ping 192.168.1.20
```

All three PCs are in the same /24 subnet; a default gateway is not required for same-subnet communication. The triangular switch links create redundancy, so STP prevents a Layer 2 loop.

Q3. Enable port security on Switch0 Fa0/3 connected to PC0.

```
enable
configure terminal
interface fa0/3
 switchport mode access
 switchport port-security
 switchport port-security maximum 1
 switchport port-security violation shutdown
 switchport port-security mac-address sticky
end

show port-security interface fa0/3
```

Sticky MAC learning stores the first allowed source MAC when traffic arrives. Before the PC sends a frame, the secure MAC table may be empty.

Q4. Check Fa0/3 port-security status before and after PC0 sends a packet.

```
! Before traffic:
show port-security interface fa0/3

! From PC0, send a packet:
ping 192.168.1.20

! Check again on Switch0:
show port-security interface fa0/3
show port-security address
```

The PDF reports the initial port as Secure-up, maximum MAC addresses 1, violation mode Shutdown, violation counter 0. After traffic, PC0 source MAC appears as the last source address. The PDF text and screenshot description are not fully consistent about the sticky MAC table count; use the live Packet Tracer output for the actual learned entry.

Q5. Replace PC0 with a different PC and check port status/violation counter.

- Disconnect PC0 from Switch0 Fa0/3.
- Connect PC-A to Fa0/3.
- Generate traffic from PC-A, for example ping 192.168.1.20.
- Because PC-A has a different MAC, port security detects an unauthorized MAC.

```
show port-security interface fa0/3
show interfaces status
```

Expected: Port Status Secure-shutdown; Security Violation Count = 1; show interfaces status displays Fa0/3 as err-disabled.

Q6. Check communication while Fa0/3 is err-disabled.

```
! From PC-A:
ping 192.168.1.20

! On Switch0:
show interfaces status
```

Expected: ping fails because the switch has disabled Fa0/3. PC-A cannot forward traffic through that port.

Q7. Recover Fa0/3 and reconnect the authorized PC0.

```
enable
configure terminal
interface fa0/3
 shutdown
 no shutdown
exit
end

! Reconnect PC0, then from PC0:
ping 192.168.1.20
```

Q8. Check STP status and identify the Root Bridge.

STP is a Layer 2 protocol that prevents switching loops in redundant topologies. Root Bridge election uses the lowest Bridge ID (priority, then MAC address as the tie-breaker). In this assignment, Switch2 is the Root Bridge.

```
! Run on Switch0, Switch1, and Switch2:
show spanning-tree
```

> On the Root Bridge output, look for: "This bridge is the root". The assignment identifies Switch2 as the Root Bridge.

Q9. Identify blocking and forwarding ports.

Switch	Port	Role	State
Switch2 (Root)	Fa0/1, Fa0/2, Fa0/3	Designated	Forwarding
Switch1	Fa0/1	Root	Forwarding
Switch1	Fa0/2, Fa0/3	Designated	Forwarding
Switch0	Fa0/1	Root	Forwarding
Switch0	Fa0/2	Alternate	Blocking
Switch0	Fa0/3	Designated	Forwarding

```
show spanning-tree
```

Q10. Remove an active triangle link and observe the previously blocked port.

- Before failure: Switch0 Fa0/1 is Root/Forwarding; Fa0/2 is Alternate/Blocking.
- Disconnect the active link on Switch0 Fa0/1.
- Run show spanning-tree on Switch0. The previously blocked Fa0/2 begins transitioning (the PDF observes Learning).

- After convergence, Fa0/2 becomes the new Root Port and Forwarding.

```
show spanning-tree
```

> Classic 802.1D STP may transition through Listening and Learning before Forwarding. The assignment screenshots specifically describe Fa0/2 as Learning during transition and Forwarding after convergence.

Q11. Change Switch1 priority to make it the Root Bridge.

```
! On Switch1:
enable
configure terminal
spanning-tree vlan 1 priority 4096
end

! Verify on all three switches:
show spanning-tree
```

The effective displayed priority is 4097 in the PDF (base 4096 plus VLAN 1 system-ID extension). Switch1 should display "This bridge is the root"; Switch0 and Switch2 should show Switch1 as Root ID.

Q12. Connect PC0, PC-A, and PC-B to a Hub, then connect the Hub to Switch0 Fa0/3.

- Remove the direct PC0-to-Switch0 Fa0/3 cable.
- Add a PT-Hub.
- Connect one Hub port to Switch0 Fa0/3 using a copper straight-through cable.
- Connect PC0, PC-A, and PC-B to separate Hub ports.
- Assign all hosts unique IP addresses in 192.168.1.0/24; the PDF examples for the new PCs are 192.168.1.40 and 192.168.1.50.

A hub is a Layer 1 repeater. It does not learn MAC addresses or filter frames; it repeats incoming signals to all other active ports.

Q13. Check the switch MAC table for the hub uplink port.

```
! Generate traffic from all PCs behind the hub, then on Switch0:
show mac address-table
```

Expected: Multiple dynamic MAC address entries (PC0, PC-A, PC-B) mapped to the same physical switch port Fa0/3.

Q14. Observe hub flooding in Simulation mode.

- Change Packet Tracer from Realtime to Simulation mode.
- Set the event filter to ICMP.
- From PC0, ping PC1: ping 192.168.1.20.
- Click Capture/Forward step by step.
- The Hub repeats the incoming frame to PC-A, PC-B, and Switch0. PC-A and PC-B discard it because the destination MAC is not theirs; Switch0 forwards it toward PC1.

Q15. Why does a switch learn multiple MAC addresses on one port when a hub is connected?

A switch learns the source MAC address of every incoming Ethernet frame and associates it with the receiving physical port. With a direct PC connection, normally one PC MAC is learned on that port. With a hub, frames from multiple PCs arrive through the same switch port, so multiple source MAC addresses are mapped to Fa0/3.

Q16. Compare collision domains: one Hub versus three separate switch ports.

Hub connection	Separate switch ports
PC0, PC-A, and PC-B share one collision domain.	Each PC-to-switch link is its own collision domain (three domains).
Half-duplex shared medium; simultaneous transmissions can collide.	Full-duplex links can send and receive simultaneously without collisions.
CSMA/CD is relevant for shared half-duplex Ethernet.	CSMA/CD is not needed on full-duplex switched links.

WEEK 8 - Static Routing

Source assignment: Week 8 Cisco Static Routing Assignment. The two-page PDF contains four topology diagrams and LAN subnet/host-count labels, but it does not print router WAN addresses or exact router interface names. The WAN IP allocations below are illustrative worked examples so the static-route syntax can be practised. Match actual interface names and cabling in Packet Tracer.

Static route syntax

```
ip route DESTINATION_NETWORK SUBNET_MASK NEXT_HOP_IP
```

Meaning: tell this router that to reach the destination network, forward packets to the next-hop router. The next-hop IP must be reachable through a directly connected interface.

> For each router, add a route for every remote LAN that is not directly connected. Directly connected networks appear automatically in the routing table as C routes.

Q1. One router connecting two LANs.

Network	Router gateway	Example hosts	Mask
LAN 1: 172.31.1.0/24	172.31.1.1	172.31.1.2, .3	255.255.255.0
LAN 2: 172.31.2.0/24	172.31.2.1	172.31.2.2, .3	255.255.255.0

```
enable
configure terminal
interface gigabitEthernet 0/0
 ip address 172.31.1.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet 0/1
 ip address 172.31.2.1 255.255.255.0
 no shutdown
exit
end
write memory

show ip interface brief
show ip route

! No ip route command is needed: both LANs are directly connected.
```

Q2. Two routers, one LAN behind each router.

> Illustrative WAN link for this worked example: 10.0.0.0/30, Router0=10.0.0.1, Router1=10.0.0.2. The uploaded Week 8 diagram does not specify these WAN addresses.

Router0

```
enable
configure terminal
interface gigabitEthernet 0/0
 ip address 172.31.1.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet 0/1
 ip address 10.0.0.1 255.255.255.252
 no shutdown
exit
ip route 172.31.2.0 255.255.255.0 10.0.0.2
end
write memory
```

Router1

```
enable
configure terminal
interface gigabitEthernet 0/0
 ip address 172.31.2.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet 0/1
 ip address 10.0.0.2 255.255.255.252
 no shutdown
exit
ip route 172.31.1.0 255.255.255.0 10.0.0.1
end
write memory
```

Q3. Two routers, three LANs (two LANs on Router0, one LAN on Router1).

Illustrative WAN link: 10.0.0.0/30. The LAN ranges/host counts are visible in the diagram: 172.31.1.0, 172.31.2.0, and 172.31.3.0. Interface/IP plan below follows the analogous Week 9 three-LAN topology.

Router0 - two LANs and WAN

```
enable
configure terminal
interface gigabitEthernet 0/2
 ip address 172.31.1.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet 0/1
 ip address 172.31.2.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet 0/0
 ip address 10.0.0.1 255.255.255.252
 no shutdown
exit
ip route 172.31.3.0 255.255.255.0 10.0.0.2
end
write memory
```

Router1 - one LAN and WAN

```
enable
configure terminal
interface gigabitEthernet 0/0
 ip address 10.0.0.2 255.255.255.252
 no shutdown
exit
interface gigabitEthernet 0/1
 ip address 172.31.3.1 255.255.255.0
 no shutdown
exit
ip route 172.31.1.0 255.255.255.0 10.0.0.1
ip route 172.31.2.0 255.255.255.0 10.0.0.1
end
write memory
```

Q4. Three routers connected in series, four LANs.

Illustrative WAN allocations: Router0-Router1 = 10.0.0.0/30 (.1/.2); Router1-Router2 = 10.0.0.4/30 (.5/.6). LANs follow the diagram: 172.31.1.0/24 through 172.31.4.0/24.

Router0 - LAN 1, LAN 2, WAN 1

```
enable
configure terminal
interface gigabitEthernet 0/2
 ip address 172.31.1.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet 0/1
 ip address 172.31.2.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet 0/0
 ip address 10.0.0.1 255.255.255.252
 no shutdown
exit
ip route 172.31.3.0 255.255.255.0 10.0.0.2
ip route 172.31.4.0 255.255.255.0 10.0.0.2
end
write memory
```

Router1 - LAN 3, WAN 1, WAN 2

```
enable
configure terminal
interface gigabitEthernet 0/0
 ip address 10.0.0.2 255.255.255.252
 no shutdown
exit
interface gigabitEthernet 0/2
 ip address 172.31.3.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet 0/1
 ip address 10.0.0.5 255.255.255.252
 no shutdown
exit
ip route 172.31.1.0 255.255.255.0 10.0.0.1
ip route 172.31.2.0 255.255.255.0 10.0.0.1
ip route 172.31.4.0 255.255.255.0 10.0.0.6
end
write memory
```

Router2 - LAN 4, WAN 2

```
enable
configure terminal
interface gigabitEthernet 0/0
 ip address 10.0.0.6 255.255.255.252
 no shutdown
exit
interface gigabitEthernet 0/1
 ip address 172.31.4.1 255.255.255.0
 no shutdown
exit
ip route 172.31.1.0 255.255.255.0 10.0.0.5
ip route 172.31.2.0 255.255.255.0 10.0.0.5
ip route 172.31.3.0 255.255.255.0 10.0.0.5
end
write memory
```

Static routing verification

```
show ip interface brief
show ip route
ping DESTINATION_IP
tracert DESTINATION_IP
```

Why does the next /30 WAN network start at 10.0.0.4?

A /30 subnet uses mask 255.255.255.252. Block size = 256 - 252 = 4 in the last octet. Therefore consecutive /30 networks start at .0, .4, .8, .12, and so on.

/30 network	Usable addresses	Broadcast
10.0.0.0/30	10.0.0.1, 10.0.0.2	10.0.0.3
10.0.0.4/30	10.0.0.5, 10.0.0.6	10.0.0.7
10.0.0.8/30	10.0.0.9, 10.0.0.10	10.0.0.11
10.0.0.12/30	10.0.0.13, 10.0.0.14	10.0.0.15

WEEK 9 - Dynamic Routing: Directly Connected & RIPv2

Source assignment: Week 9 Dynamic Routing. Q1 uses one router and two directly connected LANs. Q2-Q4 use RIPv2. The LAN, WAN, and interface addresses in this section are taken from the assignment text.

Q1. One router, two directly connected LANs (no dynamic routing needed).

Device/interface	IP address	Mask	Gateway
Router0 Fa0/0	172.31.1.1	255.255.255.0	-
Router0 Fa1/0	172.31.2.1	255.255.255.0	-
PC0	172.31.1.2	255.255.255.0	172.31.1.1
PC1	172.31.1.3	255.255.255.0	172.31.1.1
Laptop0	172.31.2.2	255.255.255.0	172.31.2.1
Laptop1	172.31.2.3	255.255.255.0	172.31.2.1

```
enable
configure terminal
interface fastEthernet 0/0
 ip address 172.31.1.1 255.255.255.0
 no shutdown
exit
interface fastEthernet 1/0
 ip address 172.31.2.1 255.255.255.0
 no shutdown
exit
end
write memory

show ip route
show ip interface brief
! Test from PC1:
ping 172.31.2.2
```

Both subnets appear as directly connected C routes. No static route or dynamic routing process is required.

RIPv2 quick theory and command pattern

RIP is a distance-vector routing protocol. Its metric is hop count: 15 is the maximum usable hop count; 16 means unreachable. RIPv2 supports classless routing and uses UDP port 520. The assignment uses version 2 and disables automatic summarization.

```
router rip
version 2
no auto-summary
network NETWORK_ADDRESS
exit
```

Q2. RIPv2 between two routers and two LANs.

Device/interface	IP address	Mask
Router0 Fa0/0	172.31.1.1	255.255.255.0
Router0 Fa1/0	10.0.0.1	255.255.255.252
Router1 Fa0/0	172.31.2.1	255.255.255.0
Router1 Fa1/0	10.0.0.2	255.255.255.252

Router0

```
enable
configure terminal
interface fastEthernet 0/0
 ip address 172.31.1.1 255.255.255.0
 no shutdown
exit
interface fastEthernet 1/0
 ip address 10.0.0.1 255.255.255.252
 no shutdown
exit
router rip
 version 2
 no auto-summary
 network 172.31.1.0
 network 10.0.0.0
exit
end
write memory
```

Router1

```
enable
configure terminal
interface fastEthernet 0/0
 ip address 172.31.2.1 255.255.255.0
 no shutdown
exit
interface fastEthernet 1/0
 ip address 10.0.0.2 255.255.255.252
 no shutdown
exit
router rip
 version 2
 no auto-summary
 network 172.31.2.0
 network 10.0.0.0
exit
end
write memory
```

Verify with show ip route rip and bidirectional ping between PC1 172.31.1.3 and Laptop1 172.31.2.3.

Q3. RIPv2 between two routers with three LANs.

Router2 / Router0 - LAN1, LAN2, WAN

```
enable
configure terminal
interface gigabitEthernet 0/2
 ip address 172.31.1.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet 0/1
 ip address 172.31.2.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet 0/0
 ip address 10.0.0.1 255.255.255.252
 no shutdown
exit
router rip
 version 2
 no auto-summary
 network 172.31.1.0
 network 172.31.2.0
 network 10.0.0.0
exit
end
write memory
```

Router3 / Router1 - LAN3, WAN

```
enable
configure terminal
interface gigabitEthernet 0/1
 ip address 172.31.3.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet 0/0
 ip address 10.0.0.2 255.255.255.252
 no shutdown
exit
router rip
 version 2
 no auto-summary
 network 172.31.3.0
 network 10.0.0.0
exit
end
write memory
```

PC/Laptop gateways: 172.31.1.1, 172.31.2.1, 172.31.3.1. Verification: show ip interface brief; show ip route rip; ping across LANs; tracert 172.31.3.3.

Q4. RIPv2 between three routers, four LANs and two WAN links.

Router0 - LAN1, LAN2, WAN to Router1

```
enable
configure terminal
interface gigabitEthernet 0/2
 ip address 172.31.1.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet 0/1
 ip address 172.31.2.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet 0/0
 ip address 10.0.0.1 255.255.255.252
 no shutdown
exit
router rip
 version 2
 no auto-summary
 network 172.31.1.0
 network 172.31.2.0
 network 10.0.0.0
exit
end
write memory
```

Router1 - WAN1, LAN3, WAN2

```
enable
configure terminal
interface gigabitEthernet 0/0
 ip address 10.0.0.2 255.255.255.252
 no shutdown
exit
interface gigabitEthernet 0/2
 ip address 172.31.3.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet 0/1
 ip address 10.0.0.5 255.255.255.252
 no shutdown
exit
router rip
 version 2
 no auto-summary
 network 10.0.0.0
 network 172.31.3.0
exit
end
write memory
```

Router2 - WAN2, LAN4

```
enable
configure terminal
interface gigabitEthernet 0/0
 ip address 10.0.0.6 255.255.255.252
 no shutdown
exit
interface gigabitEthernet 0/1
 ip address 172.31.4.1 255.255.255.0
 no shutdown
exit
router rip
 version 2
 no auto-summary
 network 10.0.0.0
 network 172.31.4.0
exit
end
write memory
```

The 10.0.0.0 RIP network statement activates the router interfaces in that major network, including the two /30 WAN links on Router1. The assignment verifies 172.31.1.0/24 through 172.31.4.0/24 using show ip route rip, ping and tracert.

RIPv2 verification commands

```
show ip interface brief
show ip route
show ip route rip
show ip protocols
ping DESTINATION_IP
tracert DESTINATION_IP
```

SUPPLEMENT - OSPF Versions of the Week 9 Topologies

The original Week 9 assignment uses RIPv2. The following OSPF configurations are supplementary conversions using the same Week 9 IP addressing. They are not original PDF commands.

OSPF essentials

OSPF (Open Shortest Path First) is a link-state routing protocol. It calculates shortest paths using cost and the SPF/Dijkstra algorithm. Common Cisco IPv4 OSPF commands are router ospf PROCESS_ID and network NETWORK WILDCARD area AREA_ID. Area 0 is the backbone area used in these examples.

Subnet mask	Wildcard mask
255.255.255.0 (/24)	0.0.0.255
255.255.255.252 (/30)	0.0.0.3

Wildcard mask is the inverse of the subnet mask. OSPF network statements select local interfaces; they are not remote-destination routes.

OSPF for Week 9 Q2 - two routers.

Router0 - AS-independent OSPF process

```
router ospf 1
 network 172.31.1.0 0.0.0.255 area 0
 network 10.0.0.0 0.0.0.3 area 0
exit
```

Router1

```
router ospf 1
 network 172.31.2.0 0.0.0.255 area 0
 network 10.0.0.0 0.0.0.3 area 0
exit
```

OSPF for Week 9 Q3 - two routers, three LANs.

Router2 / Router0

```
router ospf 1
 network 172.31.1.0 0.0.0.255 area 0
 network 172.31.2.0 0.0.0.255 area 0
 network 10.0.0.0 0.0.0.3 area 0
exit
```

Router3 / Router1

```
router ospf 1
 network 172.31.3.0 0.0.0.255 area 0
 network 10.0.0.0 0.0.0.3 area 0
exit
```

OSPF for Week 9 Q4 - three routers, four LANs.

Router0

```
router ospf 1
 network 172.31.1.0 0.0.0.255 area 0
 network 172.31.2.0 0.0.0.255 area 0
 network 10.0.0.0 0.0.0.3 area 0
exit
```

Router1 - middle router

```
router ospf 1
 network 172.31.3.0 0.0.0.255 area 0
 network 10.0.0.0 0.0.0.3 area 0
 network 10.0.0.4 0.0.0.3 area 0
exit
```

Router2

```
router ospf 1
 network 172.31.4.0 0.0.0.255 area 0
 network 10.0.0.4 0.0.0.3 area 0
exit
```

OSPF verification

```
show ip ospf neighbor
show ip route ospf
show ip route
show ip protocols
show ip interface brief
ping DESTINATION_IP
tracert DESTINATION_IP
```

Expected OSPF-learned routes are marked with O in the IPv4 routing table. Check neighbor state; it should reach FULL for a normal adjacency.

SUPPLEMENT - BGP Versions of the Week 9 Topologies

The original Week 9 assignment uses RIPv2. These are supplementary eBGP examples based on the same Week 9 IP addresses. AS numbers 65001, 65002, and 65003 are example private AS numbers chosen for this guide; they are not supplied in the original assignment.

BGP essentials

BGP (Border Gateway Protocol) is a path-vector routing protocol used to exchange routes between autonomous systems. eBGP peers belong to different AS numbers; iBGP peers belong to the same AS. BGP uses TCP port 179. Cisco IOS command pattern: router bgp AS_NUMBER, neighbor PEER_IP remote-as PEER_AS, and network PREFIX mask SUBNET_MASK.

Property	BGP
Type	Path Vector
eBGP administrative distance	20
iBGP administrative distance	200
Transport	TCP port 179
Routing-table code	B
Key attributes	AS-PATH and other path attributes

BGP for Week 9 Q2 - two routers.

Router0 - AS 65001

```
router bgp 65001
 neighbor 10.0.0.2 remote-as 65002
 network 172.31.1.0 mask 255.255.255.0
exit
```

Router1 - AS 65002

```
router bgp 65002
 neighbor 10.0.0.1 remote-as 65001
 network 172.31.2.0 mask 255.255.255.0
exit
```

BGP for Week 9 Q3 - two routers, three LANs.

Router2 / Router0 - AS 65001

```
router bgp 65001
 neighbor 10.0.0.2 remote-as 65002
 network 172.31.1.0 mask 255.255.255.0
 network 172.31.2.0 mask 255.255.255.0
exit
```

Router3 / Router1 - AS 65002

```
router bgp 65002
 neighbor 10.0.0.1 remote-as 65001
 network 172.31.3.0 mask 255.255.255.0
exit
```

BGP for Week 9 Q4 - three routers, four LANs.

Router0 - AS 65001

```
router bgp 65001
 neighbor 10.0.0.2 remote-as 65002
 network 172.31.1.0 mask 255.255.255.0
 network 172.31.2.0 mask 255.255.255.0
exit
```

Router1 - middle router, AS 65002

```
router bgp 65002
 neighbor 10.0.0.1 remote-as 65001
 neighbor 10.0.0.6 remote-as 65003
 network 172.31.3.0 mask 255.255.255.0
exit
```

Router2 - AS 65003

```
router bgp 65003
 neighbor 10.0.0.5 remote-as 65002
 network 172.31.4.0 mask 255.255.255.0
exit
```

BGP configuration and verification notes

- BGP peers must have IP reachability between the neighbor addresses before the session can establish.
- The network statement originates a prefix only when an exact matching route (prefix and mask) exists in the local routing table.
- The Q4 middle router has two eBGP neighbors: 10.0.0.1 in AS 65001 and 10.0.0.6 in AS 65003.
- Some Packet Tracer router images may not support every BGP feature; use a supported IOS router image for the lab.

```
show ip bgp summary
show ip bgp
show ip route bgp
show ip route
show ip protocols
ping DESTINATION_IP
```

RIP vs OSPF vs BGP - quick comparison

Feature	RIPv2	OSPF	BGP
Protocol type	Distance Vector	Link State	Path Vector
Metric / selection	Hop count	Cost	Path attributes; AS-PATH is important
Command start	router rip	router ospf 1	router bgp 65001
Key config	version 2; no auto-summary	network ... wildcard area 0	neighbor ... remote-as ...
Route code	R	O	B
Administrative distance	120	110	eBGP 20 / iBGP 200
Transport	UDP 520	IP protocol 89	TCP 179

WEEK 10 - VLANs, Trunk vs Access & Router-on-a-Stick

Q1. Configure five VLANs on two Cisco 2960 switches and compare trunk mode with access mode.

Topology: Switch A and Switch B; five PCs on each switch. End-device ports Fa0/1-Fa0/5 map to VLANs 10, 20, 30, 40, 50. The inter-switch link is GigabitEthernet0/1 on both switches.

VLAN	Name	PC-A IP	PC-B IP	Mask
10	CSED	192.168.10.1	192.168.10.2	255.255.255.0
20	ECE	192.168.20.1	192.168.20.2	255.255.255.0
30	CIVIL	192.168.30.1	192.168.30.2	255.255.255.0
40	ACADEMICS	192.168.40.1	192.168.40.2	255.255.255.0
50	ACCOUNTS	192.168.50.1	192.168.50.2	255.255.255.0

Run on BOTH Switch A and Switch B: create VLANs and assign PC access ports

```
enable
configure terminal
vlan 10
 name CSED
vlan 20
 name ECE
vlan 30
 name CIVIL
vlan 40
 name ACADEMICS
vlan 50
 name ACCOUNTS
exit

interface fastEthernet 0/1
 switchport mode access
 switchport access vlan 10
exit
interface fastEthernet 0/2
 switchport mode access
 switchport access vlan 20
exit
interface fastEthernet 0/3
 switchport mode access
 switchport access vlan 30
exit
interface fastEthernet 0/4
 switchport mode access
 switchport access vlan 40
exit
interface fastEthernet 0/5
 switchport mode access
 switchport access vlan 50
exit
end
write memory
```

Case A - trunk inter-switch link (on both switches)

```
enable
configure terminal
interface gigabitEthernet 0/1
 switchport mode trunk
 no shutdown
exit
end

show interfaces trunk
show vlan brief
```

Trunk mode carries multiple VLANs over one physical link. In the assignment, same-VLAN pairs across switches (VLAN 10, VLAN 20, etc.) can communicate, but different VLANs still need Layer 3 routing.

Case B - access inter-switch link assigned to VLAN 10 (on both switches)

```
configure terminal
interface gigabitEthernet 0/1
 switchport mode access
 switchport access vlan 10
exit
end

show vlan brief
```

Access mode carries only the assigned VLAN. In the assignment, VLAN 10 PC-A1 to PC-B1 succeeds; VLAN 20 PC-A2 to PC-B2 fails across the inter-switch link. The same VLAN 10 and VLAN 20 PCs on one switch are still separate VLANs and do not inter-route without a Layer 3 device.

Q2. Router-on-a-Stick using three VLANs and three switches.

Topology: Switch0 (left edge), Switch1 (right edge), Switch2 (central trunk hub), Router-PT, and PCs PC0-PC5. Router subinterfaces provide the VLAN default gateways.

VLAN	Name	PCs	Host IPs	Gateway
10	ACADEMICS	PC0, PC3	192.168.10.2, .3	192.168.10.1
20	ACCOUNTS	PC1, PC4	192.168.20.2, .3	192.168.20.1
30	OFFICE	PC2, PC5	192.168.30.2, .3	192.168.30.1

Router-PT subinterfaces (physical Fa0/0)

```
enable
configure terminal
interface fastEthernet 0/0
 no shutdown
exit
interface fastEthernet 0/0.10
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.0
exit
interface fastEthernet 0/0.20
 encapsulation dot1Q 20
 ip address 192.168.20.1 255.255.255.0
exit
interface fastEthernet 0/0.30
 encapsulation dot1Q 30
 ip address 192.168.30.1 255.255.255.0
exit
end
write memory
```

Central Switch2 - create VLANs; trunk to router and both edge switches

```
enable
configure terminal
vlan 10
 name ACADEMICS
vlan 20
 name ACCOUNTS
vlan 30
 name OFFICE
exit
interface fastEthernet 0/1
 switchport mode trunk
exit
interface fastEthernet 0/2
 switchport mode trunk
exit
interface fastEthernet 0/3
 switchport mode trunk
exit
end
write memory
```

Edge Switch0 and Switch1 - same VLAN and port pattern

```
enable
configure terminal
vlan 10
 name ACADEMICS
vlan 20
 name ACCOUNTS
vlan 30
 name OFFICE
exit
interface fastEthernet 0/1
 switchport mode access
 switchport access vlan 10
exit
interface fastEthernet 0/2
 switchport mode access
 switchport access vlan 20
exit
interface fastEthernet 0/3
 switchport mode access
 switchport access vlan 30
exit
interface fastEthernet 0/4
 switchport mode trunk
exit
end
write memory
```

Physical port mapping follows the assignment topology: Switch2 Fa0/1 to Router, Fa0/2 to Switch0, Fa0/3 to Switch1; each edge switch Fa0/4 is its uplink. Verify your Packet Tracer cable endpoints.

Verification and test commands

```
! On Router:
show ip route
show ip interface brief

! On Switch2:
show interfaces trunk
show vlan brief

! From PC0:
ping 192.168.20.2
ping 192.168.30.3
```

The assignment reports an initial ARP timeout on the first ping followed by successful replies, and successful PC0-to-PC5 communication across VLANs.

WEEK 11 - Inter-VLAN Routing with SVIs, ROAS & Trunk Failure

Q1. Inter-VLAN routing using SVIs across four access switches and one multilayer switch.

Topology: one Multilayer Switch (3560), four access switches (2960), and eight PCs. VLANs are 10, 11, 12, 13, and 15. VLAN 15 is shared by one PC on each access switch.

PC	VLAN	IP	Gateway
PC0	10	192.168.10.2	192.168.10.1
PC1	15	192.168.15.2	192.168.15.1
PC2	11	192.168.11.2	192.168.11.1
PC3	15	192.168.15.3	192.168.15.1
PC4	12	192.168.12.2	192.168.12.1
PC5	15	192.168.15.4	192.168.15.1
PC6	13	192.168.13.2	192.168.13.1
PC7	15	192.168.15.5	192.168.15.1

Multilayer Switch (3560)

```
enable
configure terminal
ip routing

vlan 10
vlan 11
vlan 12
vlan 13
vlan 15
exit

interface range gigabitEthernet 1/0/1 - 4
 switchport mode trunk
 no shutdown
exit

interface vlan 10
 ip address 192.168.10.1 255.255.255.0
 no shutdown
exit
interface vlan 11
 ip address 192.168.11.1 255.255.255.0
 no shutdown
exit
interface vlan 12
 ip address 192.168.12.1 255.255.255.0
 no shutdown
exit
interface vlan 13
 ip address 192.168.13.1 255.255.255.0
 no shutdown
exit
interface vlan 15
 ip address 192.168.15.1 255.255.255.0
 no shutdown
exit
end
write memory
```

Access switch template - repeat per switch, changing the VLAN IDs for the local PC ports

```
! Example Switch0: PC0 in VLAN 10, PC1 in VLAN 15
 enable
 configure terminal
 vlan 10
 vlan 15
 exit
 interface fastEthernet 0/1
  switchport mode access
  switchport access vlan 10
  no shutdown
 exit
 interface fastEthernet 0/2
  switchport mode access
  switchport access vlan 15
  no shutdown
 exit
 interface fastEthernet 0/3
  switchport mode trunk
  no shutdown
 exit
 end
 write memory

! Repeat the pattern on other access switches:
! Switch1: VLAN 11 on Fa0/1, VLAN 15 on Fa0/2, Fa0/3 trunk
! Switch2: VLAN 12 on Fa0/1, VLAN 15 on Fa0/2, Fa0/3 trunk
! Switch3: VLAN 13 on Fa0/1, VLAN 15 on Fa0/2, Fa0/3 trunk
```

Exact VLAN/port commands for the remaining access switches

Switch1 - local VLAN 11 and shared VLAN 15

```
enable
configure terminal
vlan 11
vlan 15
exit
interface fastEthernet 0/1
 switchport mode access
 switchport access vlan 11
 no shutdown
exit
interface fastEthernet 0/2
 switchport mode access
 switchport access vlan 15
 no shutdown
exit
interface fastEthernet 0/3
 switchport mode trunk
 no shutdown
exit
end
write memory
```

Switch2 - local VLAN 12 and shared VLAN 15

```
enable
configure terminal
vlan 12
vlan 15
exit
interface fastEthernet 0/1
 switchport mode access
 switchport access vlan 12
 no shutdown
exit
interface fastEthernet 0/2
 switchport mode access
 switchport access vlan 15
 no shutdown
exit
interface fastEthernet 0/3
 switchport mode trunk
 no shutdown
exit
end
write memory
```

Switch3 - local VLAN 13 and shared VLAN 15

```
enable
configure terminal
vlan 13
vlan 15
exit
interface fastEthernet 0/1
 switchport mode access
 switchport access vlan 13
 no shutdown
exit
interface fastEthernet 0/2
 switchport mode access
 switchport access vlan 15
 no shutdown
exit
interface fastEthernet 0/3
 switchport mode trunk
 no shutdown
exit
end
write memory
```

Verification

```
show vlan brief
show ip interface brief
show ip route
show interfaces trunk

! Example cross-VLAN ping:
ping 192.168.11.2
```

Q2. Compare SVI routing on a multilayer switch with Router-on-a-Stick (four VLANs).

VLANs: 10 Sales, 20 HR, 30 IT, 40 Finance. PC0-PC3 use .2 addresses in their respective /24 networks; each gateway is .1.

VLAN	Name	PC IP	Gateway
10	Sales	192.168.10.2	192.168.10.1
20	HR	192.168.20.2	192.168.20.1
30	IT	192.168.30.2	192.168.30.1
40	Finance	192.168.40.2	192.168.40.1

Access switch - create VLANs, assign Fa0/1-Fa0/4, trunk Fa0/5

```
enable
configure terminal
vlan 10
 name Sales
vlan 20
 name HR
vlan 30
 name IT
vlan 40
 name Finance
exit
interface range fastEthernet 0/1 - 4
 switchport mode access
 no shutdown
exit
interface fastEthernet 0/1
 switchport access vlan 10
exit
interface fastEthernet 0/2
 switchport access vlan 20
exit
interface fastEthernet 0/3
 switchport access vlan 30
exit
interface fastEthernet 0/4
 switchport access vlan 40
exit
interface fastEthernet 0/5
 switchport mode trunk
 no shutdown
exit
end
write memory
```

Method A - Multilayer Switch (3560) SVI

```
enable
configure terminal
ip routing
vlan 10
vlan 20
vlan 30
vlan 40
exit
interface fastEthernet 0/1
 switchport mode trunk
 no shutdown
exit
interface vlan 10
 ip address 192.168.10.1 255.255.255.0
 no shutdown
exit
interface vlan 20
 ip address 192.168.20.1 255.255.255.0
 no shutdown
exit
interface vlan 30
 ip address 192.168.30.1 255.255.255.0
 no shutdown
exit
interface vlan 40
 ip address 192.168.40.1 255.255.255.0
 no shutdown
exit
end
write memory
```

Method B - Router-on-a-Stick (router physical interface shown as Gi0/0/0 in the assignment concept)

```
enable
configure terminal
interface gigabitEthernet 0/0/0
 no shutdown
exit
interface gigabitEthernet 0/0/0.10
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.0
exit
interface gigabitEthernet 0/0/0.20
 encapsulation dot1Q 20
 ip address 192.168.20.1 255.255.255.0
exit
interface gigabitEthernet 0/0/0.30
 encapsulation dot1Q 30
 ip address 192.168.30.1 255.255.255.0
exit
interface gigabitEthernet 0/0/0.40
 encapsulation dot1Q 40
 ip address 192.168.40.1 255.255.255.0
exit
end
write memory
```

Router interface names vary by model (for example Gi0/0/0, Gi0/0, or Fa0/0). Select the actual physical interface connected to the trunk, then create subinterfaces under that exact name.

SVI vs Router-on-a-Stick

Parameter	Multilayer SVI	Router-on-a-Stick
Routing device	Layer 3 switch	Router
Logical gateway	interface Vlan10 etc.	PhysicalInterface.10 etc.
VLAN tagging	Trunk link required	encapsulation dot1Q on each subinterface
Routing command	ip routing	Router routes between connected subnets
Typical use	Larger LANs	Smaller networks/labs
Potential bottleneck	Integrated Layer 3 switching	Shared router trunk

Verification

```
show vlan brief
show ip route
show ip interface brief
show interfaces trunk

! From PC0, example pings:
ping 192.168.20.2
ping 192.168.30.2
ping 192.168.40.2
```

Q3. VLAN 10 and VLAN 20 across two access switches with a multilayer switch; test trunk failure.

PC	Switch	VLAN	IP	Gateway
PC0	Switch0	10	192.168.10.2	192.168.10.1
PC1	Switch0	20	192.168.20.2	192.168.20.1
PC2	Switch1	10	192.168.10.3	192.168.10.1
PC3	Switch1	20	192.168.20.3	192.168.20.1

Switch0 (2960)

```
enable
configure terminal
vlan 10
 name Sales
vlan 20
 name HR
exit
interface fastEthernet 0/1
 switchport mode access
 switchport access vlan 10
 no shutdown
exit
interface fastEthernet 0/2
 switchport mode access
 switchport access vlan 20
 no shutdown
exit
interface fastEthernet 0/3
 switchport mode trunk
 no shutdown
exit
end
write memory
```

Switch1 (2960)

```
enable
configure terminal
vlan 10
 name Sales
vlan 20
 name HR
exit
interface fastEthernet 0/1
 switchport mode access
 switchport access vlan 10
 no shutdown
exit
interface fastEthernet 0/2
 switchport mode access
 switchport access vlan 20
 no shutdown
exit
interface fastEthernet 0/3
 switchport mode trunk
 no shutdown
exit
end
write memory
```

Multilayer Switch 3560

```
enable
configure terminal
ip routing
vlan 10
 name Sales
vlan 20
 name HR
exit
interface fastEthernet 0/1
 switchport mode trunk
 no shutdown
exit
interface fastEthernet 0/2
 switchport mode trunk
 no shutdown
exit
interface vlan 10
 ip address 192.168.10.1 255.255.255.0
 no shutdown
exit
interface vlan 20
 ip address 192.168.20.1 255.255.255.0
 no shutdown
exit
end
write memory
```

Verification and fault test

```
! On Switch0 and Switch1:
show vlan brief

! On MLS:
show interfaces trunk
show ip route
show ip interface brief

! From PC0 before fault:
ping 192.168.10.1
ping 192.168.10.3
ping 192.168.20.2
ping 192.168.20.3

! Fault test on MLS: shut down trunk to Switch0
configure terminal
interface fastEthernet 0/1
 shutdown
end
show interfaces trunk
```

After the MLS Fa0/1 trunk to Switch0 is shut down: PC0-to-PC1 fails; PC0-to-PC2 fails; PC0-to-PC3 fails; PC2-to-PC3 remains successful because Switch1 trunk Fa0/2 remains active.

WEEK 12 - UDP Socket Programming

Source assignment: Network Lab Assignment Week 12 and 13. This section preserves the UDP Echo and UDP Time-of-Day program structure from the PDF, with compile/run commands for a Linux terminal.

Socket programming quick reference

Concept	UDP	TCP
Socket type	SOCK_DGRAM	SOCK_STREAM
Server setup	socket, bind	socket, bind, listen, accept
Send	sendto	send
Receive	recvfrom	recv
Connection	Connectionless	Connection-oriented
Address per message	sendto/recvfrom carry peer address	Connected socket after connect/accept

Compile and run pattern

```
# Compile a C program:
gcc -Wall -Wextra FileName.c -o ProgramName

# Start the server in Terminal 1:
./ProgramName

# Run the client in Terminal 2:
./ClientProgram server_ip arguments
```

Q1. UDP Echo Server and Client

Server listens on UDP port 8888, receives one datagram, prints it, sends the same data back, and exits. Client accepts a server IP and a quoted string; string length must be 1-32 bytes inclusive.

Compile and run

```
gcc -Wall -Wextra EchoServer.c -o EchoServer
gcc -Wall -Wextra EchoClient.c -o EchoClient

# Terminal 1:
./EchoServer

# Terminal 2:
./EchoClient 127.0.0.1 "Hello UDP"
```

UDP Echo Server - EchoServer.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8888
#define MAXLINE 1024

int main(void) {
    int sockfd;
    char buffer[MAXLINE];
    struct sockaddr_in servaddr, cliaddr;
    socklen_t len;

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    memset(&servaddr, 0, sizeof(servaddr));
    memset(&cliaddr, 0, sizeof(cliaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = INADDR_ANY;
    servaddr.sin_port = htons(PORT);

    if (bind(sockfd, (const struct sockaddr *)&servaddr,
             sizeof(servaddr)) < 0) {
        perror("Bind failed");
        close(sockfd);
        exit(EXIT_FAILURE);
    }

    printf("Server started ... waiting for connection ...\n");
    fflush(stdout);

    len = sizeof(cliaddr);
    int n = recvfrom(sockfd, buffer, MAXLINE - 1, 0,
                     (struct sockaddr *)&cliaddr, &len);
    if (n < 0) {
        perror("Receive failed");
        close(sockfd);
        exit(EXIT_FAILURE);
    }
    buffer[n] = '\0';
    printf("Received from client: %s\n", buffer);

    sendto(sockfd, buffer, strlen(buffer), 0,
           (const struct sockaddr *)&cliaddr, len);

    close(sockfd);
    return 0;
}
```

UDP Echo Client - EchoClient.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8888
#define MAXLINE 1024

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s <Server IP> \"<string>\"\n", argv[0]);
        return 1;
    }

    char *server_ip = argv[1];
    char *input_string = argv[2];
    size_t str_len = strlen(input_string);
    if (str_len < 1 || str_len > 32) {
        printf("Error: Input string length out of range (1 to 32 bytes).\n");
        return 1;
    }

    int sockfd;
    char buffer[MAXLINE];
    struct sockaddr_in servaddr;
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    memset(&servaddr, 0, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    if (inet_pton(AF_INET, server_ip, &servaddr.sin_addr) <= 0) {
        printf("Error: Invalid Server IP address.\n");
        close(sockfd);
        return 1;
    }

    sendto(sockfd, input_string, str_len, 0,
           (const struct sockaddr *)&servaddr, sizeof(servaddr));

    socklen_t len = sizeof(servaddr);
    int n = recvfrom(sockfd, buffer, MAXLINE - 1, 0,
                     (struct sockaddr *)&servaddr, &len);
    if (n < 0) {
        perror("Receive failed");
        close(sockfd);
        exit(EXIT_FAILURE);
    }
    buffer[n] = '\0';
    printf("Received from server: %s\n", buffer);

    close(sockfd);
    return 0;
}
```

Q2. UDP Time-of-Day Server and Client

Client sends REQUEST_TIME_OF_DAY to UDP server on port 8888. Server reads local system time using time()/ctime(), removes the trailing newline, and sends the time string back.

Compile and run

```
gcc -Wall -Wextra TimeOfDayServer.c -o TimeOfDayServer
gcc -Wall -Wextra TimeOfDayClient.c -o TimeOfDayClient

# Terminal 1:
./TimeOfDayServer

# Terminal 2:
./TimeOfDayClient 127.0.0.1 8888
```

UDP Time-of-Day Server - TimeOfDayServer.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <time.h>
#include <arpa/inet.h>

#define PORT 8888
#define MAXLINE 1024

int main(void) {
    int sockfd;
    char buffer[MAXLINE];
    struct sockaddr_in servaddr, cliaddr;
    socklen_t len;

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    memset(&servaddr, 0, sizeof(servaddr));
    memset(&cliaddr, 0, sizeof(cliaddr));

    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = INADDR_ANY;
    servaddr.sin_port = htons(PORT);

    if (bind(sockfd, (const struct sockaddr *)&servaddr,
             sizeof(servaddr)) < 0) {
        perror("Bind failed");
        close(sockfd);
        exit(EXIT_FAILURE);
    }

    printf("TimeOfDay Server started ... waiting for client request ...\n");
    fflush(stdout);

    len = sizeof(cliaddr);
    int n = recvfrom(sockfd, buffer, MAXLINE - 1, 0,
                     (struct sockaddr *)&cliaddr, &len);
    if (n < 0) {
        perror("Receive failed");
        close(sockfd);
        exit(EXIT_FAILURE);
    }
    buffer[n] = '\0';
    printf("Received request from client [%s:%d]: \"%s\"\n",
           inet_ntoa(cliaddr.sin_addr), ntohs(cliaddr.sin_port), buffer);

    time_t rawtime;
    time(&rawtime);
    char *time_str = ctime(&rawtime);
    time_str[strcspn(time_str, "\n")] = 0;

    printf("Sending Time of Day response: %s\n", time_str);
    sendto(sockfd, time_str, strlen(time_str), 0,
           (const struct sockaddr *)&cliaddr, len);

    close(sockfd);
    printf("Server request fulfilled. Connection closed.\n");
    return 0;
}
```

UDP Time-of-Day Client - TimeOfDayClient.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define MAXLINE 1024

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s <Server IP> <Port Number>\n", argv[0]);
        return 1;
    }

    char *server_ip = argv[1];
    int port = atoi(argv[2]);
    int sockfd;
    char buffer[MAXLINE];
    struct sockaddr_in servaddr;

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    memset(&servaddr, 0, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(port);

    if (inet_pton(AF_INET, server_ip, &servaddr.sin_addr) <= 0) {
        printf("Error: Invalid Server IP address.\n");
        close(sockfd);
        return 1;
    }

    char *request_msg = "REQUEST_TIME_OF_DAY";
    printf("Sending request message to server %s:%d ...\n", server_ip, port);
    sendto(sockfd, request_msg, strlen(request_msg), 0,
           (const struct sockaddr *)&servaddr, sizeof(servaddr));

    socklen_t len = sizeof(servaddr);
    int n = recvfrom(sockfd, buffer, MAXLINE - 1, 0,
                     (struct sockaddr *)&servaddr, &len);
    if (n < 0) {
        perror("Receive failed");
        close(sockfd);
        exit(EXIT_FAILURE);
    }
    buffer[n] = '\0';
    printf("Received from server: Time of Day = %s\n", buffer);

    close(sockfd);
    return 0;
}
```

WEEK 13 - TCP Socket Programming

Q1. TCP String Reversal Server and Client

Client connects to TCP server on port 8080, enters strings, sends them to the server, and receives the reversed string. The client exits when the user enters exit or quit.

Compile and run

```
gcc -Wall -Wextra StringReversalServer.c -o StringReversalServer
gcc -Wall -Wextra StringReversalClient.c -o StringReversalClient

# Terminal 1:
./StringReversalServer

# Terminal 2:
./StringReversalClient 127.0.0.1 8080
```

TCP String Reversal Server - StringReversalServer.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

void reverse_string(char *str) {
    int n = (int)strlen(str);
    for (int i = 0; i < n / 2; i++) {
        char temp = str[i];
        str[i] = str[n - i - 1];
        str[n - i - 1] = temp;
    }
}

int main(void) {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int opt = 1;
    socklen_t addrlen = sizeof(address);
    char buffer[BUFFER_SIZE];

    /* socket() returns -1 on failure; use < 0 for the error check. */
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR,
                   &opt, sizeof(opt)) < 0) {
        perror("setsockopt failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }

    memset(&address, 0, sizeof(address));
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("Bind failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }
    if (listen(server_fd, 3) < 0) {
        perror("Listen failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }

    printf("TCP String Reversal Server started on port %d ... waiting ...\n", PORT);
    fflush(stdout);

    if ((new_socket = accept(server_fd, (struct sockaddr *)&address,
                             &addrlen)) < 0) {
        perror("Accept failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }
    printf("Client connected: %s:%d\n",
           inet_ntoa(address.sin_addr), ntohs(address.sin_port));

    while (1) {
        memset(buffer, 0, BUFFER_SIZE);
        int valread = recv(new_socket, buffer, BUFFER_SIZE - 1, 0);
        if (valread <= 0) {
            printf("Client disconnected or closed connection.\n");
            break;
        }
        buffer[strcspn(buffer, "\r\n")] = 0;
        printf("Received string: %s\n", buffer);
        reverse_string(buffer);
        printf("Sending reversed string: %s\n", buffer);
        send(new_socket, buffer, strlen(buffer), 0);
    }

    close(new_socket);
    close(server_fd);
    return 0;
}
```

The uploaded PDF shows the TCP reversal server socket failure comparison as == 0. The standard POSIX socket() call returns -1 on failure, so the code above uses the correct < 0 test.

TCP String Reversal Client - StringReversalClient.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s <Server IP> <Port>\n", argv[0]);
        return 1;
    }

    char *server_ip = argv[1];
    int port = atoi(argv[2]);
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE];
    char input_str[BUFFER_SIZE];

    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Socket creation error");
        return -1;
    }
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(port);
    if (inet_pton(AF_INET, server_ip, &serv_addr.sin_addr) <= 0) {
        printf("Invalid address / Address not supported\n");
        close(sock);
        return -1;
    }

    printf("Connecting to Server at %s:%d ...\n", server_ip, port);
    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("Connection Failed");
        close(sock);
        return -1;
    }
    printf("Connected! Enter strings (exit or quit to stop):\n");

    while (1) {
        printf("Enter string: ");
        if (fgets(input_str, sizeof(input_str), stdin) == NULL) break;
        input_str[strcspn(input_str, "\r\n")] = 0;
        if (strlen(input_str) == 0) continue;
        if (strcmp(input_str, "exit") == 0 ||
            strcmp(input_str, "quit") == 0) {
            printf("Exiting application...\n");
            break;
        }

        printf("Sending string to server: %s\n", input_str);
        send(sock, input_str, strlen(input_str), 0);
        memset(buffer, 0, BUFFER_SIZE);
        int valread = recv(sock, buffer, BUFFER_SIZE - 1, 0);
        if (valread <= 0) {
            printf("Server closed connection.\n");
            break;
        }
        printf("Received string from server: %s\n", buffer);
    }

    close(sock);
    return 0;
}
```

Q2. TCP Arithmetic Calculator Server and Client

Client sends an expression in the format operand1 operator operand2. Valid operators are +, -, *, /, and ^. The server parses the expression, evaluates it, and sends the result or an error message. The client enters -1 to quit.

Compile and run

```
gcc -Wall -Wextra CalcServerTCP.c -o CalcServerTCP -lm
gcc -Wall -Wextra CalcClientTCP.c -o CalcClientTCP

# Terminal 1:
./CalcServerTCP 8080

# Terminal 2:
./CalcClientTCP 127.0.0.1 8080
```

TCP Calculator Server - CalcServerTCP.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <math.h>
#include <arpa/inet.h>

#define BUFFER_SIZE 1024

double evaluate(double op1, char op, double op2, int *error) {
    *error = 0;
    switch (op) {
        case '+': return op1 + op2;
        case '-': return op1 - op2;
        case '*': return op1 * op2;
        case '/':
            if (op2 == 0) {
                *error = 1;
                return 0;
            }
            return op1 / op2;
        case '^': return pow(op1, op2);
        default:
            *error = 2;
            return 0;
    }
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <Port Number>\n", argv[0]);
        return 1;
    }
    int port = atoi(argv[1]);
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len = sizeof(client_addr);
    char buffer[BUFFER_SIZE];

    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(port);

    if (bind(server_fd, (struct sockaddr *)&server_addr,
             sizeof(server_addr)) < 0) {
        perror("Bind failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }
    if (listen(server_fd, 5) < 0) {
        perror("Listen failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }

    while (1) {
        printf("Server listening on port %d\n", port);
        fflush(stdout);
        client_fd = accept(server_fd, (struct sockaddr *)&client_addr,
                           &addr_len);
        if (client_fd < 0) {
            perror("Accept failed");
            continue;
        }
        printf("Client %s on port %d connected\n",
               inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));

        while (1) {
            memset(buffer, 0, BUFFER_SIZE);
            int bytes_received = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
            if (bytes_received <= 0) {
                printf("Client closed connection\n");
                break;
            }
            buffer[strcspn(buffer, "\r\n")] = 0;
            printf("Received from client: %s\n", buffer);

            double op1, op2, result;
```

```
            char op;
            char response[BUFFER_SIZE];
            if (sscanf(buffer, "%lf %c %lf", &op1, &op, &op2) == 3) {
                int error = 0;
                result = evaluate(op1, op, op2, &error);
                if (error == 1) {
                    snprintf(response, sizeof(response),
                             "Error: Division by zero");
                } else if (error == 2) {
                    snprintf(response, sizeof(response),
                             "Error: Invalid operator");
                } else {
                    snprintf(response, sizeof(response), "%g", result);
                }
            } else {
                snprintf(response, sizeof(response),
                         "Error: Invalid expression format");
            }
            printf("Sending to client: %s\n", response);
            send(client_fd, response, strlen(response), 0);
        }
        close(client_fd);
    }
    close(server_fd);
    return 0;
}
```

TCP Calculator Client - CalcClientTCP.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <netdb.h>
#include <arpa/inet.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s <Server Hostname/IP> <Port Number>\n", argv[0]);
        return 1;
    }
    char *hostname = argv[1];
    int port = atoi(argv[2]);

    struct hostent *he = gethostbyname(hostname);
    if (he == NULL) {
        herror("Hostname resolution failed");
        return 1;
    }
    char *server_ip = inet_ntoa(*((struct in_addr *)he->h_addr_list[0]));

    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE];
    char input_expr[BUFFER_SIZE];

    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Socket creation error");
        return -1;
    }
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(port);
    serv_addr.sin_addr = *((struct in_addr *)he->h_addr_list[0]);

    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("Connection Failed");
        close(sock);
        return -1;
    }
    printf("TCP client connected to %s on port %d\n", server_ip, port);

    while (1) {
        printf("Enter expression: operand1 operator operand2 (+ - * / ^). -1 to quit.\n");
        if (fgets(input_expr, sizeof(input_expr), stdin) == NULL) break;
        input_expr[strcspn(input_expr, "\r\n")] = 0;
        if (strcmp(input_expr, "-1") == 0) {
            printf("Bye!\n");
            break;
        }
        if (strlen(input_expr) == 0) continue;

        send(sock, input_expr, strlen(input_expr), 0);
        memset(buffer, 0, BUFFER_SIZE);
        int bytes_received = recv(sock, buffer, BUFFER_SIZE - 1, 0);
        if (bytes_received <= 0) {
            printf("Server closed connection.\n");
            break;
        }
        printf("ANS: %s = %s\n", input_expr, buffer);
    }
    close(sock);
    return 0;
}
```

Socket programming command/function memory table

Function	Meaning
socket()	Create a UDP or TCP socket.
bind()	Attach a server socket to local IP/port.
listen()	Put a TCP server socket into listening state.
accept()	Accept a TCP client connection.
connect()	Connect a TCP client to a server.
sendto()/recvfrom()	Send/receive UDP datagrams with peer address.
send()/recv()	Send/receive data on a connected TCP socket.
inet_pton()	Convert presentation IP to binary network form.
inet_ntoa()	Convert IPv4 binary address to dotted-decimal string.
htons()	Convert 16-bit port to network byte order.
ntohs()	Convert 16-bit port from network to host byte order.
memset()	Initialize/clear a memory block.
strlen()	Get string length.
strcspn()	Find first character from a set; used to strip newline.
sscanf()	Parse formatted values from a string.
snprintf()	Format a string safely into a buffer.
pow()	Calculate a numeric power.

TCP is a byte stream, not a message-preserving protocol. These lab examples send small strings and assume one recv corresponds to one send. For production applications, add message framing (length prefix or delimiter) and handle partial sends/receives.

Final Rapid Revision - Commands to Memorise

VLAN and SVI

```
enable
configure terminal
ip routing
vlan 10
vlan 20
exit
interface vlan 10
 ip address 192.168.10.1 255.255.255.0
 no shutdown
exit
interface vlan 20
 ip address 192.168.20.1 255.255.255.0
 no shutdown
exit
interface fastEthernet 0/1
 switchport mode access
 switchport access vlan 10
exit
interface fastEthernet 0/2
 switchport mode access
 switchport access vlan 20
exit
end
write memory
```

Trunk and access ports

```
interface gigabitEthernet 0/1
 switchport mode trunk
 no shutdown
exit

interface fastEthernet 0/1
 switchport mode access
 switchport access vlan 10
 no shutdown
exit
```

Port security and recovery

```
interface fastEthernet 0/3
 switchport mode access
 switchport port-security
 switchport port-security maximum 1
 switchport port-security violation shutdown
 switchport port-security mac-address sticky
exit

! Recovery:
interface fastEthernet 0/3
 shutdown
 no shutdown
exit
```

STP root and priority

```
show spanning-tree

! Make Switch1 root for VLAN 1:
spanning-tree vlan 1 priority 4096
show spanning-tree
```

Static routing

```
ip route DESTINATION_NETWORK SUBNET_MASK NEXT_HOP_IP
show ip route
```

RIPv2

```
router rip
 version 2
 no auto-summary
 network NETWORK_ADDRESS
exit
show ip route rip
```

OSPF

```
router ospf 1
 network NETWORK_ADDRESS WILDCARD_MASK area 0
exit
show ip ospf neighbor
show ip route ospf
```

BGP

```
router bgp LOCAL_AS
 neighbor NEIGHBOR_IP remote-as REMOTE_AS
 network PREFIX mask SUBNET_MASK
exit
show ip bgp summary
show ip route bgp
```

Exam-ready distinctions

Question	Remember
VLAN vs SVI	VLAN creates a Layer 2 broadcast domain; SVI provides a Layer 3 gateway for that VLAN.
Access vs Trunk	Access carries one VLAN; trunk carries multiple VLANs using 802.1Q tagging.
SVI vs ROAS	SVI is on a multilayer switch; ROAS uses router subinterfaces with encapsulation dot1Q.
Static vs Dynamic	Static route is manually entered; RIP/OSPF/BGP exchange routes dynamically.
RIP vs OSPF vs BGP	RIP=hop count; OSPF=cost/SPF; BGP=path attributes/AS-PATH.
Port security	Controls secure MAC addresses on a switch port; sticky learns MAC automatically.
STP	Prevents Layer 2 loops; lowest Bridge ID becomes Root Bridge.
Hub vs Switch	Hub repeats at Layer 1; switch learns MACs and forwards at Layer 2.

Important accuracy notes

- Week 8 source PDF contains topology diagrams and LAN labels, but not explicit WAN IP addresses or router interface names. The static-routing WAN addresses in this guide are labelled illustrative.
- Week 9 original assignment uses RIPv2. OSPF and BGP sections are supplementary conversions, not original assignment answers.
- When interface commands fail, inspect the actual router/switch ports. Device models differ.
- For BGP, each neighbor must be reachable and the exact network prefix/mask must exist in the local routing table before the network statement advertises it.
- For SVIs, ensure VLANs exist and their Layer 2 ports are active; otherwise an SVI may remain down/down.
- For TCP programs, start the server before running the client. The calculator server uses pow(), so link with -lm when compiling on Linux.